Zeiterfassung on Kubernetes

SAII HS2026 · Klasse 26A · Gruppe 6
Yogani Y., Luca G., Yanick M.

1 SOLL

We want a small time-tracking service that runs on Kubernetes in a dev and a prod variant. A person picks their name from a list (a new name plus Enter creates the person), stamps `EIN`, `PAUSE`, `ZURÜCK` or `AUS` and can add a note per stamp (the "Rapport", e.g. *Support XYZ Kunde Meyer*). The main page shows a status board with every person, their current status and the time they worked today, plus the last 20 stamps and which pod answered.

1.1 functionality

- Every person has their own state. Allowed transitions are out → `EIN` → working → `PAUSE` → on break → `ZURÜCK` → working → `AUS` → out. Anything else is rejected with `409`, and buttons that aren't allowed are disabled in the UI.
- There is no login. A person is just a name, created once and then picked from the list, compared case-insensitively. Stamping a name that doesn't exist is rejected with `404`, so a typo doesn't create a second person. Real users and RBAC were optional in the brief and we left them out on purpose, so anyone can stamp for anyone.
- Stamps are stored append-only in SQLite under `/data` on a PVC. Deleting every pod doesn't lose anything, `make smoke` checks exactly that.
- The form post redirects back to the page (`303`), a JSON post returns the stamp (`201`), so the same endpoint works for the browser and for `curl`.

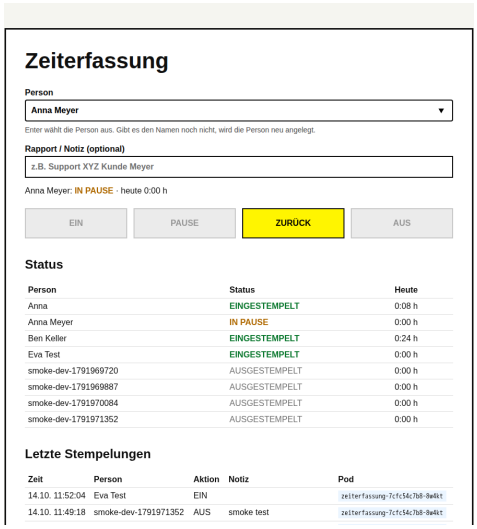


Figure 1: UI with a person selected

1.2 platform

The cluster is k3d (k3s in Docker) with a single node and a local registry next to it. One Kustomize base holds deployment, service and PVC, and the two overlays only change what really differs:

	dev	**prod**
replicas	1	3
exposure	`NodePort` 30080 → `localhost:8080`	`ClusterIP` + Traefik `Ingress` → `zeiterfassung.localhost:8081`
image	local image, `k3d image import`	pulled from the k3d registry
resources	50m/64Mi requested, 250m/128Mi limit	100m/128Mi requested, 500m/256Mi limit
namespace	`zeiterfassung-dev`	`zeiterfassung-prod`

```
app/          app.py, store.py, templates/, static/, tests/, Dockerfile
k8s/base/     deployment.yaml, service.yaml, pvc.yaml, kustomization.yaml
k8s/overlays/dev/     kustomization.yaml, namespace.yaml, service-nodeport.yaml
k8s/overlays/prod/    kustomization.yaml, namespace.yaml, ingress.yaml, resources-patch.yaml
Makefile, smoke.sh, README.md, docs/
```

Setup is `make cluster dev prod`, the README has every step as plain commands too, including the minikube equivalents. The host only needs Docker, k3d, kubectl, make and curl: the unit tests run in a `test` stage of the Dockerfile (a failing test stops the build), and this PDF is built with the Typst container.

2 Security requirements

Every requirement maps to the line that implements it. Where we decided against something, it's listed as well.

#	**Requirement**	**Implemented by**	**Threat**
1	Container runs as non-root, UID 10001	Dockerfile `USER 10001`, pod `runAsNonRoot`, `runAsUser: 10001`	breakout to host root
2	No Linux capabilities, no privilege escalation	`capabilities.drop: ["ALL"]`, `allowPrivilegeEscalation: false`	privilege abuse, setuid binaries
3	Read-only root filesystem	`readOnlyRootFilesystem: true`, only `/data` (PVC) and `/tmp` (emptyDir, 16Mi) writable	persisting injected payloads

4	Default seccomp profile	`seccompProfile: RuntimeDefault`	syscall surface
5	No service account token in the pod	`automountServiceAccountToken: false`	API access from a compromised pod
6	CPU and memory requests and limits, dev smaller than prod	`resources` in base, prod patch	resource exhaustion, noisy neighbour
7	Base image pinned by digest, no build tools at runtime	`python:3.13-slim@sha256:…`, multi-stage build, pip removed	drifting base, CVE surface
8	No known HIGH/CRITICAL CVEs	`make scan` (Trivy): 4 HIGH in pip found, pip removed, now 0	known vulnerabilities
9	Input validation	action whitelist, name ≤ 60 chars, note ≤ 200 chars, otherwise `400`; unknown person `404`	injection, unbounded writes
10	Output escaping	Jinja autoescape, tested with `<script>` in name and note	stored XSS via the note
11	Strict browser policy	`Content-Security-Policy: default-src 'none'; style-src 'self'; script-src 'self'; form-action 'self'; frame-ancestors 'none'`, no inline script, `nosniff`	XSS, clickjacking
12	Consistent data with 3 writers	status check and insert in one `BEGIN IMMEDIATE` transaction	double stamps, corrupt records
13	Prod not exposed via NodePort	prod `ClusterIP` + Ingress only	unintended exposure
14	No secrets in image, repo or prompts	none needed, `.dockerignore` is an allow list	credential leaks
–	*Not done:* authentication / RBAC	optional in the brief, see SOLL	stamping for someone else
–	*Not done:* NetworkPolicy, TLS on the ingress	lab only, worth adding before going live	lateral movement, sniffing

For us the important point is that these aren't only in the YAML. Requirements 1–3 are also why the forced failure in the next section couldn't simply `chmod /data`, the container isn't allowed to.

3 Healthchecks

`/health` is used for both probes. It doesn't just return `200`, since a check like that would stay green with a full or read-only volume. It does two things:

```
with tempfile.NamedTemporaryFile(dir=data_dir, prefix=".health-") as probe:
    probe.write(b"ok"); probe.flush()        # is /data writable?
store.ping()                                  # can SQLite open and read the database?
```

If either fails, it answers `503` with the error, otherwise `200` with pod name and version.

Probe	Delay / period	Failures	Effect
readiness	2s / 5s	2	pod is taken out of the Service, no restart
liveness	10s / 10s	3	container is restarted

Readiness starts early and reacts fast, so traffic only goes to pods that can actually stamp. Liveness waits longer and needs three failures, because a restart doesn't fix a broken volume. It only helps if the process itself is stuck. A `preStop` sleep of 5 seconds keeps a terminating pod serving until Traefik has removed it. Without it, 3 of 290 requests failed during a rollout, with it 0 of 678.

3.1 forced failure

We locked the database file in the running dev pod and watched the probe react:

```
$ kubectl -n zeiterfassung-dev exec $POD -- chmod 000 /data/zeiterfassung.db
$ kubectl -n zeiterfassung-dev get pods
NAME                           READY   STATUS    RESTARTS   AGE
zeiterfassung-78988bf6-mzkv6   0/1     Running   0          98s
$ kubectl -n zeiterfassung-dev exec $POD -- python -c "...urlopen('http://localhost:8080/
health')"
503 {"error":"unable to open database file","pod":"zeiterfassung-78988bf6-
mzkv6","status":"unhealthy"}
$ kubectl -n zeiterfassung-dev exec $POD -- chmod 644 /data/zeiterfassung.db
$ kubectl -n zeiterfassung-dev get pods          # 8 seconds later
NAME                           READY   STATUS    RESTARTS   AGE
zeiterfassung-78988bf6-mzkv6   1/1     Running   0          107s
```

From the host, `curl localhost:8080/health` got no answer while the pod wasn't ready, because the Service had no endpoint left. After restoring the file the pod was ready again without a restart.

3.2 a bug the healthcheck found in itself

The first version wrote a fixed file `.health-<pod>`. Readiness and liveness fire at the same moment every 10 seconds, both threads wrote the same file, and the second `unlink` failed. The result was a `503` every 10 seconds and one liveness restart in prod, while all unit tests were green. We only saw it in `kubectl get events`. The fix is the unique temp file above, plus a test with 200 parallel health calls that failed before and passes now. This is in the AI decision log as well.

4 Deviations and risks

Deviation	**Why, and what it costs**
k3d instead of minikube	Our setup runs k3d. Every course command has an equivalent, the README maps them. On rootless Docker k3d needs a kubelet feature gate and `--mode direct` for the image import, the Makefile handles that.
3 replicas on one `ReadWriteOnce` volume	The brief asks for both. It only works because the cluster has one node and all pods mount the same directory. A second node would break prod mounting. Protects against pod crashes, not node loss.
SQLite shared by 3 pods	Locking is reliable on one kernel and a local filesystem, verified with 20 parallel `EIN` for one person: 1× `201`, 19× `409`. On a network volume we'd move to a database pod.
Built with Claude Code, not Hermes	The decision log keeps the HowTo format with Claude Code as the AI, and only lists proposals we actually checked.
No authentication	Optional in the brief. Anyone can stamp for any name, fine for the lab, not for real working hours.